I2C SWITCH: MASTER + SLAVE

Pure bare-metal C on two NUCLEO-F446RE boards (I2C1, address 0x28)

Part 1: the story | Part 2: night-before-exam deep dive

What the program does, in one breath

Press the USER button (PC13) on the **MASTER** board. It **writes** the byte `'S'` to the slave at address `0x28`. The **SLAVE** board blinks its LED 5 times and arms an acknowledge. The master then **reads** one byte back and the slave returns `'D'`. The master blinks twice to confirm. Guard counters mean a missing slave can never hang the master.

Part	What you get	How to use it
1. The Story	The whole program as a short tale, each scene tagged to the code it represents	Read once, slowly. Builds the mental movie.
2A. Big picture	Wiring, roles, I2C theory, frame format, I2C vs SPI vs UART	Skim first on exam night.
2B. Bit-level decode	Every register line decoded to bits and hex, flag-clearing sequences	Memorize the numbers and sequences.
2C. Code walkthrough	Master write/read step by step, slave loop, full timeline	Be able to redraw the frame and timeline.
2D. Weak points + debug	Design flaws, improvements, troubleshooting table	Great for 'suggest an improvement' questions.
2E. Q and A + cram card	18 likely exam questions with model answers, one-page cheat sheet	Last 30 minutes before sleep.

How to read the callouts

EXAM TIP = something examiners love to ask. **TRAP** = a subtle behaviour that causes real bugs.

PART 1 - THE STORY

The Captain, the Guard of House 0x28, and the Party Line

Two houses stand on a quiet street. A **Captain Master** lives in one and owns a big button. A **Guard** (the Slave) lives in the house numbered 0x28 (forty, in plain numbers) and owns a lantern. They are joined by just two wires, a *party line*: one wire carries the beat, the other carries the words. Anyone may pull a wire down. Nobody can push it up. A pair of stretchy ropes (the pull-up resistors) always lift it back. That is the whole street. Here is what happens when the Captain presses his button.

Character	Real thing	Role in the story
Captain Master	The MASTER board	Owns the button (PC13); starts and ends every call
Guard of House 0x28	The SLAVE board, address 0x28	Owns the lantern (LED PA5); answers only to his house number
The beat wire	SCL = PB8	Clock. The Captain sets the pace; a slow Guard may hold it down to ask for time
The word wire	SDA = PB9	Data, shared by both directions, one speaker at a time
Stretchy ropes	4.7 kOhm pull-ups to 3.3 V	Lift both wires back to HIGH whenever nobody pulls them down
The nod	ACK (9th bit, pulled LOW)	'Heard you, go on'
The head-shake	NACK (9th bit stays HIGH)	'Nobody home' or 'that is enough'
Secret words	`'S'` = 0x53, `'D'` = 0x44	Request word and Done word

Chapter 1 - Wiring the street

Power first: the Captain switches on three buildings (GPIOA for the lantern, GPIOB for the wires, GPIOC for the button) and the telephone exchange, I2C1. The Guard needs only two buildings. Then the two wires are given their special jobs. They become I2C wires (alternate function 4), and, crucially, **open-drain**: they may only pull down, never push up, so two voices can never fight and short-circuit. The exchange is then tuned to the street's speed: a 16 MHz wristwatch, a half-beat of 80 ticks, and a rope-lift allowance of 17 ticks. That works out to a calm 100 kHz.

[code: RCC enables, MODER, OTYPER, PUPDR, AFRH, I2C1_CR2 / CCR / TRISE, PE]

Chapter 2 - The Guard gets a house number

Unlike a guard in a world of private pipes, this Guard does not wait for someone to drum at him. He is given a **house number** (0x28) in his registry, turns on his habit of nodding, and sits listening to the party line. Hundreds of conversations could pass by; he reacts only when he hears his own number. The moment he does, a flag goes up (ADDR) and the beat wire is held low until he acknowledges it.

[code: I2C1_OAR1 = (0x28 << 1) | (1<<14); CR1 |= ACK | PE; wait ADDR]

Chapter 3 - The call that says WRITE

The Captain presses his button. He debounces for 20 ms, then checks the party line is quiet (not BUSY). He shouts **START** by pulling the word wire down while the beat wire is high. Then he calls the house number followed by one extra bit that announces his intent: **0 means 'I will WRITE to you'**. The Guard nods (ACK). The Captain speaks `'S'`. The Guard nods again. The Captain announces **STOP** by releasing the word wire while the beat wire is high. If nobody had nodded, the Captain would simply give up and walk home: his patience counters (the GUARD constants) make sure he never waits forever.

[code: i2c_write(0x28, 'S')]

Chapter 4 - The Guard does his duty

The Guard reads `'S'`, waits for the STOP, then lights his lantern five times (two seconds). Afterwards he puts a note in his pocket: 'ack ready'.

[code: if (b == 'S') { blink(5); ack_ready = 1; }]

Chapter 5 - The call that says READ

Like in the last kingdom, the Captain cannot see the Guard's progress, so he waits 2.5 seconds by the clock. Then he calls again: START, house number, and this time the extra bit is **1, meaning 'I want to READ from you'**. The Guard nods and, because the bit told him the direction, he knows *right now* that he must speak. He consults his pocket note and says `'D'`. The Captain, wanting only one byte, shakes his head instead of nodding (NACK) which means 'that is enough', and announces STOP. The Guard notices the head-shake (flag AF), clears it, and tears up the pocket note. The Captain heard `'D'`, so he flashes his lantern twice: mission confirmed.

[code: i2c_read(0x28, &ack;); slave: TRA=1, I2C1_DR = 'D'; wait AF]

THE MORAL - three laws of I2C hidden in this story

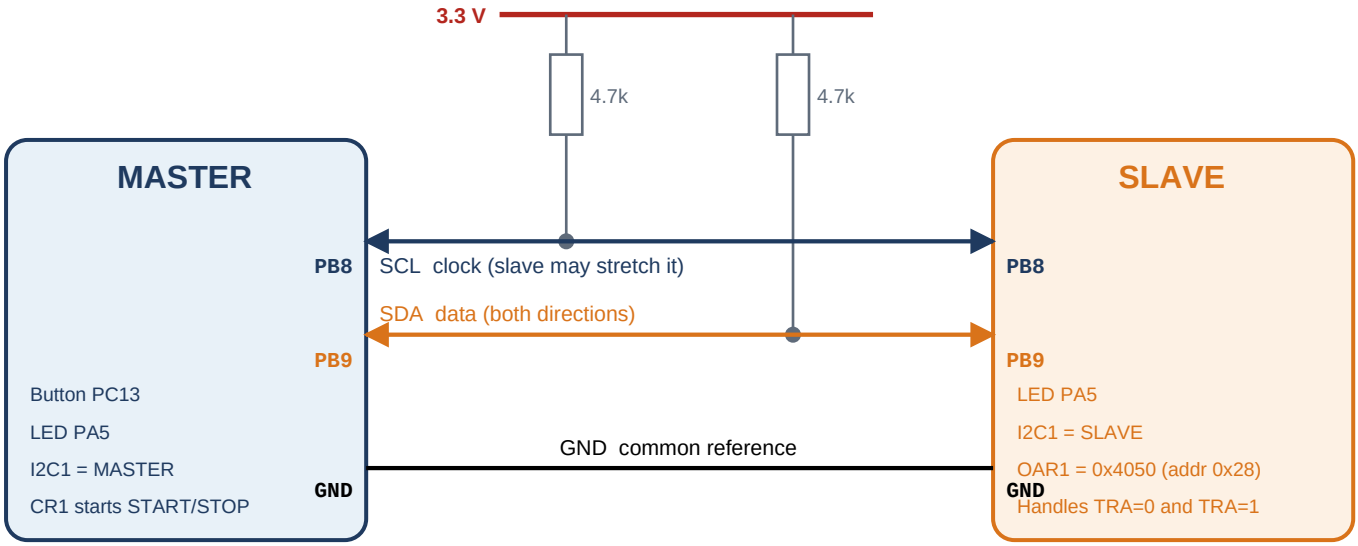
1) **The address picks the listener**, not a wire: two wires can serve many devices.
2) **Direction is announced up front** (the R/W bit), so the slave builds its reply on demand. There is no 'preloaded envelope that arrives one transfer late' as in SPI.
3) **Every byte is answered by a nod (ACK) or head-shake (NACK)**. That one bit is both error detection and flow control.

PART 2 - NIGHT-BEFORE-EXAM DEEP DIVE

2A. Big picture

60-SECOND SUMMARY

Two NUCLEO-F446RE boards on I2C1: PB8 = SCL, PB9 = SDA (AF4, open-drain), 4.7k pull-ups to 3.3 V, common GND. 100 kHz standard mode (CR2 = 16, CCR = 80, TRISE = 17). Slave address 0x28 (OAR1 = 0x4050). Press -> master WRITE 'S' (0x53) -> slave blinks x5 (2000 ms) and arms ack -> master waits 2500 ms -> master READ one byte -> slave sends 'D' (0x44) -> master NACKs, STOPs -> master blinks x2. Master uses guard counters so a missing slave only makes the call fail.



Both SCL pins connect to each other and both SDA pins connect to each other (straight, not crossed). Only ONE pair of pull-ups is needed on the whole bus; the code also enables the chips' internal pull-ups (about 40 kOhm each) but those are too weak to rely on.

Roles side by side

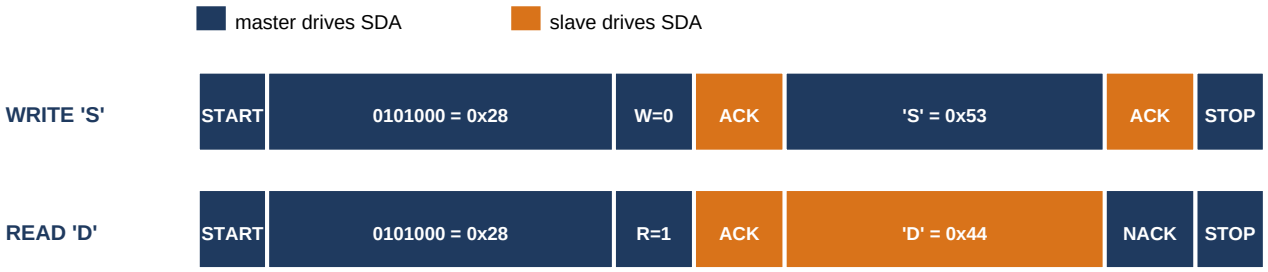
Item	MASTER project	SLAVE project
Clocks enabled	GPIOA, GPIOB, GPIOC, I2C1	GPIOA, GPIOB, I2C1 (no button)
PA5	Output, LED (2 confirm blinks)	Output, LED (5 request blinks)
PB8 / PB9	AF4, open-drain, pull-up (SCL, SDA)	Same configuration
Address	Calls 0x28 (sends 0x50 for write, 0x51 for read)	OAR1 = 0x4050 (ADD = 0x28, bit 14 = 1)
CR1 usage	PE, START, STOP, ACK toggled per transfer, SWRST pulse	ACK + PE (0x401), SWRST pulse
Timing regs	CR2 = 16, CCR = 80, TRISE = 17	Same values (set while PE = 0)
Main loop	Polls button every 5 ms; one write + one read per press	Waits for ADDR; branches on TRA (receive or transmit)
Initiates?	YES, always	NEVER; only reacts when addressed

I2C theory you must be able to write from memory

- **2 signals:** SCL (clock) and SDA (data), both open-drain with pull-up resistors (wired-AND: any device can pull the line low; high only if everyone lets go).
- **START:** SDA falls while SCL is high. **STOP:** SDA rises while SCL is high. Otherwise SDA may change only while SCL is low.
- **Frame:** START, 7-bit address, R/W bit, ACK, data byte(s) each followed by ACK/NACK, STOP. 9 clock pulses per byte.

- **R/W bit:** 0 = master writes to slave, 1 = master reads from slave. Address 0x28 becomes 0x50 (write) or 0x51 (read).
- **ACK / NACK:** the receiver pulls SDA low on the 9th clock for ACK. The master NACKs the last byte it reads to tell the slave to stop sending.
- **Half duplex, addressed, multi-device:** one speaker at a time; each device has an address; multi-master is possible through arbitration.
- **Clock stretching:** a slave may hold SCL low to make the master wait. The F446 does this automatically while ADDR is unserviced.
- **Standard mode:** 100 kHz. Fast mode: 400 kHz. This design uses standard mode: SCL period 10 us, one byte (9 clocks) = 90 us.

What a transaction looks like on the wire



Address byte on the wire: 0x50 for write, 0x51 for read (7-bit address shifted left, R/W in bit 0)

I2C vs SPI vs UART (always asked)

Feature	I2C	SPI	UART
Wires	2 (SDA, SCL) shared by all devices	3 + one chip-select per slave	2 (TX, RX) per link
Clock	Synchronous (master SCL)	Synchronous (master SCK)	Asynchronous, agreed baud
Selecting a slave	7-bit address in the frame	Hardware CS line (or none, as in the SPI demo)	Point to point
Duplex	Half	Full	Full
Output stage	Open-drain + pull-ups	Push-pull	Push-pull
Acknowledge	ACK/NACK after every byte	None built in	None built in
Slave reply	On demand, after the R/W bit	Preloaded, arrives one transfer late	n/a
Flow control	Clock stretching	None	Optional RTS/CTS
Typical speed	100 k / 400 k / 1 MHz	Several MHz to tens of MHz	9600 to ~1 Mbaud

2B. Bit-level decode of every register line

Memory map (base + offset = address)

Macro in code	Address	Base + offset	Notes
RCC_AHB1ENR	0x40023830	RCC 0x40023800 + 0x30	GPIO clock enables
RCC_APB1ENR	0x40023840	RCC + 0x40	bit 21 = I2C1EN
GPIOA_MODER / ODR	0x40020000 / 0x40020014	GPIOA + 0x00 / + 0x14	LED PA5
GPIOB_MODER	0x40020400	GPIOB 0x40020400 + 0x00	pin mode
GPIOB_OTYPER	0x40020404	GPIOB + 0x04	push-pull (0) or open-drain (1)
GPIOB_PUPDR	0x4002040C	GPIOB + 0x0C	pulls
GPIOB_AFRH	0x40020424	GPIOB + 0x24	AF number for pins 8-15
GPIOC_PUPDR / IDR	0x4002080C / 0x40020810	GPIOC 0x40020800 + 0x0C / + 0x10	button PC13
I2C1_CR1	0x40005400	I2C1 0x40005400 + 0x00	control 1
I2C1_CR2	0x40005404	I2C1 + 0x04	FREQ field
I2C1_OAR1	0x40005408	I2C1 + 0x08	own address (slave)
I2C1_DR	0x40005410	I2C1 + 0x10	data / address register
I2C1_SR1	0x40005414	I2C1 + 0x14	status 1 (event flags)
I2C1_SR2	0x40005418	I2C1 + 0x18	status 2 (MSL, BUSY, TRA)
I2C1_CCR	0x4000541C	I2C1 + 0x1C	clock control
I2C1_TRISE	0x40005420	I2C1 + 0x20	max rise time

Clock enables

Line	Bits set	Meaning
`RCC_AHB1ENR |= bits 0,1,2`	bit0, bit1, bit2	GPIOAEN, GPIOBEN, GPIOCEN. Slave sets only bits 0 and 1.
`RCC_APB1ENR |= (1<<21)`	bit21	I2C1EN. A peripheral with its clock off ignores register writes.

GPIO configuration with mask math

Pin n: `MODER` at bits [2n+1:2n]; `OTYPER` bit n (1 = open-drain); `PUPDR` at [2n+1:2n]; `AFRH` for pins 8-15 at [4(n-8)+3 : 4(n-8)]. PB8 sits in AFRH bits 3:0, PB9 in bits 7:4.

Register	Clear mask	Set value	Result
GPIOB_MODER (PB8, PB9)	(3<<16)|(3<<18) = 0x000F0000	(2<<16)|(2<<18) = 0x000A0000	bits 19:16 = 10 10: both pins alternate function
GPIOB_OTYPER	(none needed)	(1<<8)|(1<<9) = 0x00000300	PB8, PB9 open-drain (required by I2C)
GPIOB_PUPDR	(3<<16)|(3<<18) = 0x000F0000	(1<<16)|(1<<18) = 0x00050000	internal pull-ups on (weak, about 40k)
GPIOB_AFRH	(0xF<<0)|(0xF<<4) = 0x000000FF	(4<<0)|(4<<4) = 0x00000044	PB8, PB9 = AF4 = I2C1_SCL, I2C1_SDA
GPIOA_MODER (PA5)	3<<10	1<<10 = 0x400	PA5 output (LED)
GPIOC_PUPDR (PC13)	3<<26	1<<26 = 0x04000000	button pull-up (master only)

EXAM TIP - why open-drain is mandatory for I2C

If one device drove HIGH (push-pull) while another drove LOW on the same wire, the two output stages would short the supply to ground. Open-drain avoids this: devices only pull LOW, and the pull-up resistor makes the line HIGH when released. It also enables clock stretching and multi-master arbitration.

I2C1 configuration registers

Register	Value	Meaning / calculation
I2C1_CR1 bit 15 (SWRST)	set then cleared	Software reset: puts the peripheral in a known state before configuring
I2C1_CR2 (FREQ[5:0])	16 = 0x10	Tells the peripheral its APB1 clock in MHz (16 MHz HSI). Must be at least 2
I2C1_CCR	80	Standard mode (bit 15 = 0). CCR = fPCLK1 / (2 x fSCL) = 16 MHz / (2 x 100 kHz) = 80. Thigh = Tlow = 80 x 62.5 ns = 5 us, so SCL period = 10 us = 100 kHz
I2C1_TRISE	17	Max rise time in standard mode is 1000 ns. TRISE = 1000 ns / 62.5 ns + 1 = 16 + 1 = 17
I2C1_OAR1 (slave)	0x4050	ADD[7:1] = 0x28 -> 0x28 << 1 = 0x50. Bit 14 must be kept at 1 by software. Bit 15 = 0 = 7-bit addressing
I2C1_CR1 |= PE (bit 0)	master: 0x01	Enable the peripheral LAST, after CR2/CCR/TRISE (they require PE = 0)
I2C1_CR1 |= ACK | PE (slave)	0x401	bit 10 = ACK (acknowledge own address and received bytes) + bit 0 = PE

I2C1_CR1 bits used

Bit	Name	Who uses it	Meaning
0	PE	both	Peripheral enable
8	START	master	Generate START (hardware clears it when sent). Needs bus free
9	STOP	master	Generate STOP after the current byte
10	ACK	both	1 = send ACK after a received byte/address; 0 = send NACK
15	SWRST	both	Software reset (pulse 1 then 0)

Status flags (the heart of the exam)

Reg.bit	Flag	Set when	Cleared by	Used where
SR1.0	**SB**	START sent (master)	Read SR1, then write DR	after START
SR1.1	**ADDR**	Master: address ACKed. Slave: own address matched	Read SR1, then read SR2	master EV6, slave EV1
SR1.2	**BTF**	Byte transfer finished (data shifted, DR empty/full)	Read SR1 then read/write DR	master write: last byte
SR1.4	**STOPF**	STOP detected (slave)	Read SR1, then write CR1	slave after receive
SR1.6	**RXNE**	Receive data register not empty	Read DR	slave rx, master read
SR1.7	**TXE**	Transmit data register empty	Write DR	write data / slave tx
SR1.10	**AF**	Acknowledge failure (NACK received)	Write 0 to the bit	master: no slave; slave: master NACKed
SR2.0	MSL	Master mode	-	not used here
SR2.1	**BUSY**	Bus is busy (START seen, no STOP yet)	hardware	master checks before START
SR2.2	**TRA**	Transmitter (1) or receiver (0)	hardware	slave decides direction

TRAP - why ADDR is cleared by reading SR1 THEN SR2

The hardware only clears ADDR after the exact sequence: read SR1, then read SR2. In the slave code the polling condition `while(!(I2C1_SR1&2))` already read SR1, so the following `sr2 = I2C1_SR2` completes the sequence and also tells the slave the direction (TRA). Reading SR2 alone would NOT clear it.

2C. Code walkthrough

Boilerplate both files share

```
void SystemInit(void) __attribute__((weak));        void SystemInit(void) { }
void __libc_init_array(void) __attribute__((weak)); void __libc_init_array(void) { }
```

Empty weak stubs satisfy the startup file's calls to `SystemInit` and `__libc_init_array`. SystemInit being empty leaves the clock at reset defaults (HSI 16 MHz), which is what CR2 = 16, CCR = 80 and the delay loops assume. `delay_ms` is a software busy-wait of about 1 ms per outer iteration.

Master helpers

Item	Meaning
`SLAVE_ADDR 0x28`	7-bit address, shifted left by 1 when sent. Bit 0 is the R/W bit
`GUARD 200000`	Loop budget for each wait. If a flag never arrives, the loop ends and the function fails instead of hanging
`pressed()`	Button is active-low: PC13 reads 0 when pressed
`blink_little()`	2 x (120 ms ON + 120 ms OFF) = 480 ms confirm

i2c_write(addr, data) step by step (master transmitter)

#	Code	Waits for / event	What happens on the bus
1	wait SR2.BUSY == 0	bus free	Do not start if another transfer is running
2	CR1 |= START (bit 8)	-	START condition generated
3	wait SR1.SB	EV5	START sent; guard fail -> STOP, return 0
4	DR = (addr<<1)|0	-	Address byte 0x50 goes out (R/W = 0). Writing DR clears SB
5	wait SR1.ADDR or AF	EV6	ADDR = slave ACKed. AF = NACK, nobody home
6	if AF: clear, STOP, return 0	failure path	Missing slave handled cleanly
7	read SR1, read SR2	clears ADDR	Master transmitter now ready to send data
8	wait TXE; DR = data	EV8_1 / EV8	'S' (0x53) clocked out, slave ACKs
9	wait SR1.BTF	EV8_2	Last byte fully shifted out
10	CR1 |= STOP (bit 9); return 1	-	STOP condition. Success

i2c_read(addr, *out) step by step (master receiver, ONE byte)

#	Code	Waits for / event	Why
1	CR1 |= ACK	-	Start with ACK on (address phase)
2	wait not BUSY; START; wait SB	EV5	Same start sequence as write
3	DR = (addr<<1)|1	-	Address byte 0x51 (R/W = 1)
4	wait ADDR or AF	EV6	Failure path same as write
5	**CR1 &= ~ACK**	before clearing ADDR	NACK the only byte we want, so the slave stops sending
6	read SR1, read SR2	clears ADDR	Receiving begins
7	**CR1 |= STOP**	right after clearing ADDR	STOP is sent right after the byte arrives
8	wait RXNE; *out = DR	EV7	Read the byte ('D' = 0x44)

TRAP - the single-byte read recipe order matters

For exactly 1 byte: (a) clear ACK **before** clearing ADDR, (b) clear ADDR (read SR1, SR2), (c) set STOP **immediately**, (d) wait RXNE and read DR. Setting ACK=0 late would make the slave think more bytes are wanted.

Master main loop

```
int prev = 0;
while (1) {
    int now = pressed();
    if (now && !prev) {                          // NEW press only (edge detect)
        delay_ms(20);                            // debounce
        if (pressed()) {
            if (i2c_write(SLAVE_ADDR, 'S')) {    // 1 = slave ACKed address
                delay_ms(2500);                  // blind wait: slave blinks 2000 ms
                unsigned char ack = 0;
                if (i2c_read(SLAVE_ADDR, &ack) && ack == 'D')
                    blink_little();              // confirmed
            }
        }
    }
    prev = now;
    delay_ms(5);
}
```

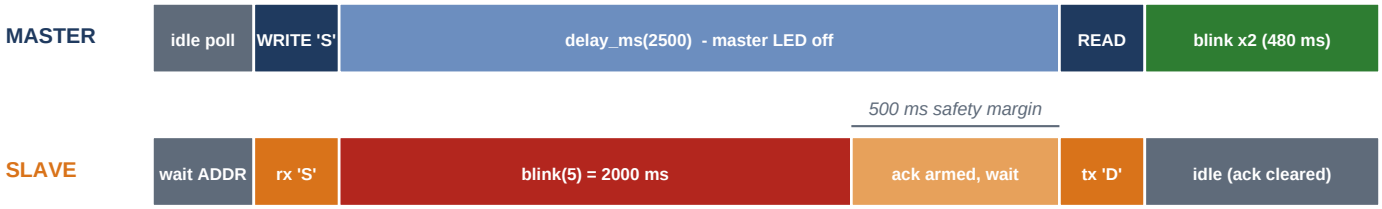
Slave main loop, annotated

```
I2C1_OAR1 = (MY_ADDR << 1) | (1 << 14);          // own address 0x28, bit 14 must be 1
I2C1_CR2 = 16;  I2C1_CCR = 80;  I2C1_TRISE = 17; // timing (while PE = 0)
I2C1_CR1 |= (1 << 10) | (1 << 0);                // ACK on + enable

while (1) {
    while (!(I2C1_SR1 & (1 << 1)));              // ADDR: someone said our number
    unsigned int sr2 = I2C1_SR2;                 // SR1 then SR2 read -> ADDR cleared; gives TRA

    if (!(sr2 & (1 << 2))) {                     // TRA = 0: master WRITES to us
        while (!(I2C1_SR1 & (1 << 6)));          //   RXNE
        unsigned char b = (unsigned char)I2C1_DR;
        while (!(I2C1_SR1 & (1 << 4)));          //   STOPF
        (void)I2C1_SR1; I2C1_CR1 |= (1 << 0);    //   clear STOPF: read SR1, write CR1
        if (b == 'S') { blink(5); ack_ready = 1; }
    } else {                                     // TRA = 1: master READS from us
        while (!(I2C1_SR1 & (1 << 7)));          //   TXE
        I2C1_DR = ack_ready ? 'D' : 0x00;        //   reply on demand
        while (!(I2C1_SR1 & (1 << 10)));         //   AF: master NACKed after 1 byte
        I2C1_SR1 &= ~(1 << 10);                  //   clear AF
        ack_ready = 0;
    }
}
```

EXAM TIP - SPI vs I2C reply timing

In SPI the slave's reply had to be preloaded **before** the transfer and arrived one transfer late. Here the master's R/W bit tells the slave about the read *first*, so the slave loads DR **during** the transaction and the 'D' comes back in the same transaction. The `ack_ready` variable remembers that the blink is done.

Full timeline of one button press



(segment widths NOT to scale)

#	When	Master	Bus	Slave
0	Power-up	Init, polls button every 5 ms	SCL, SDA both HIGH (pull-ups)	Init, waits for ADDR
1	Press	Edge seen, 20 ms debounce, re-check	idle	waiting
2	t0	i2c_write: START, 0x50, 'S' (0x53), STOP	Both ACKs by slave; about 200 us	ADDR matched, TRA=0, RXNE, STOPF, reads 'S'
3	t0 .. t0+2000 ms	delay_ms(2500)	idle	blink(5); then ack_ready = 1
4	t0+2000 ms	still waiting	idle	back at loop top, waiting for ADDR
5	t0+2500 ms	i2c_read: START, 0x51, reads 1 byte, NACK, STOP	Slave ACKs address, sends 0x44	ADDR, TRA=1, TXE, DR = 'D', then AF
6	after	ack == 'D' -> blink_little() (480 ms)	idle	AF cleared, ack_ready = 0
7	idle	back to polling	idle	back to waiting

Timing numbers to memorize

Quantity	Value	How
SCL frequency	100 kHz	16 MHz / (2 x 80)
SCL period / byte	10 us / 90 us	1 / 100 kHz; 9 clocks per byte (8 data + ACK)
One write transaction	about 200 us	START + address byte + data byte + STOP
Slave blink time	2000 ms	5 x (200 ON + 200 OFF), BLINK_MS = 200
Master wait	2500 ms	Fixed delay_ms(2500)
Safety margin	500 ms	2500 - 2000
Master success blink	480 ms	2 x (120 + 120)
Guard	200000 loop passes	Each wait loop abandons after this many iterations

2D. Weak points, improvements, debugging

Design weaknesses (great for 'how would you improve this?')

Weakness	Why it matters	Better design
Blind 2500 ms delay	Timing is guessed. If the slave blinks longer, the master's read fails or times out.	Slave returns a status byte (busy / done) and the master polls; or rely on clock stretching with a longer guard.
Slave blocks inside blink()	For 2 s it cannot service the bus. The hardware will stretch SCL if addressed, but the master's guard may expire first.	Use the I2C event interrupt and a non-blocking blink (timer or state machine).
Slave loops have no timeouts	If the master aborts after the address byte, the slave waits forever on RXNE or STOPF. If the master never NACKs, it waits forever on AF.	Add guard counters or a watchdog to the slave too.
Master ignores some guard results	Waits for TXE and BTF in i2c_write end without checking whether the guard expired, so the code continues blindly.	Check each wait and abort with STOP plus return 0.
No bus-recovery routine	If a slave holds SDA low (stuck mid-byte), BUSY stays set and every later call fails.	SWRST pulse, or bit-bang up to 9 SCL pulses, then STOP.
Polling everywhere	CPU is busy-waiting; blink delays are software loops.	Interrupts or DMA, SysTick for delays.
Minimal protocol	One command byte, one ack byte; reading without a prior 'S' returns 0x00.	Define commands, status codes, optional CRC.
Internal pull-ups	About 40 kOhm is too weak for reliable timing.	Use external 4.7 kOhm (as wired) and drop the internal ones.

Troubleshooting table

Symptom	Likely cause	Check
Nothing happens at all	Missing pull-ups; SDA/SCL swapped or crossed; no common GND	PB8 to PB8 (SCL), PB9 to PB9 (SDA); 4.7k from each to 3.3 V
Slave never blinks	Address NACK: slave not flashed or address mismatch; PE or ACK not set; OAR1 bit 14 missing	MY_ADDR = SLAVE_ADDR = 0x28; OAR1 = 0x4050; CR1 = 0x401
Slave blinks, master never confirms	Read failed or timed out; slave blinked longer than 2.5 s; ack_ready not set	BLINK_MS x 10 must stay below 2500; check master read guard
Works first press, fails after	Bus stuck BUSY (a slave holding SDA low) or slave stuck waiting in a loop	Reset both boards; add recovery and slave timeouts
Master hangs	Not hanging forever by design (guards); long pauses mean guard expiring repeatedly	Look for SB, ADDR never arriving: wiring or pull-ups
Intermittent behaviour	Weak pull-ups, long jumpers, shared ground missing	Short wires, external 4.7k, common GND
Reset order	Slave powered after master	Start the slave first; a missing slave simply makes the call fail

2E. Likely exam questions with model answers

Q1. Why must I2C lines be open-drain with pull-ups?

A. Open-drain lets any device pull a line low but never drive it high, so two devices can never short supply to ground. The pull-up resistor raises the line when everyone releases it, which also enables clock stretching and arbitration (wired-AND).

Q2. Why are the internal pull-ups not enough?

A. They are about 40 kOhm, too weak to meet the rise-time limit at 100 kHz. The design relies on external 4.7 kOhm resistors to 3.3 V; the internal ones are redundant.

Q3. How is the 100 kHz SCL frequency produced?

A. Standard mode: CCR = fPCLK1 / (2 x fSCL) = 16 MHz / 200 kHz = 80. Thigh = Tlow = 80 x 62.5 ns = 5 us, so the SCL period is 10 us.

Q4. What are CR2.FREQ = 16 and TRISE = 17?

A. FREQ gives the peripheral its APB1 clock in MHz (16). TRISE = max rise time / TPCLK1 + 1 = 1000 ns / 62.5 ns + 1 = 17 (standard mode allows 1000 ns).

Q5. What is on the wire for address 0x28?

A. The 7-bit address 0101000 followed by the R/W bit: 0x50 (write) or 0x51 (read). The slave's ACK follows on the 9th clock.

Q6. Define START and STOP.

A. START: SDA goes high to low while SCL is high. STOP: SDA goes low to high while SCL is high. Data may change only while SCL is low.

Q7. Why does the master clear ACK (NACK) when reading one byte?

A. The master must tell the slave that this is the last byte; a NACK on the 9th clock does that, so the slave releases SDA and the master can send STOP. Here the slave sees it as the AF flag.

Q8. Why is the order ACK=0, clear ADDR, STOP important for 1-byte reads?

A. ACK must be off before the first byte's 9th clock; clearing ADDR starts reception; STOP must be programmed right away so it follows that single byte. Doing it later risks receiving a second byte.

Q9. How is ADDR cleared and why do both SR1 and SR2 get read?

A. ADDR is cleared by reading SR1 followed by reading SR2. In the slave, the polling loop read SR1 and `sr2 = I2C1_SR2` reads SR2, which also provides TRA.

Q10. How does the slave know whether to receive or transmit?

A. SR2.TRA: 0 = receiver (master wrote), 1 = transmitter (master reads). The hardware sets it from the R/W bit in the address byte.

Q11. How do you clear STOPF, SB and AF?

A. STOPF: read SR1, then write CR1. SB: read SR1, then write DR. AF: write 0 to SR1 bit 10.

Q12. What does the master do if the slave is missing?

A. The address gets no ACK, so AF sets. The code clears AF, sets STOP and returns 0; the main loop skips the delay and read. The guard counters also stop it hanging if SB or ADDR never arrive.

Q13. What is OAR1 = 0x4050?

A. (0x28 << 1) = 0x50 puts the 7-bit address in bits 7:1. Bit 14 must be kept at 1 by software (reference manual rule). Bit 15 = 0 selects 7-bit addressing.

Q14. What is clock stretching and where can it happen in this design?

A. A slave may hold SCL low to make the master wait. The F446 does it automatically after the address matches until firmware clears ADDR, and in the transmitter until DR is written. The slave's long blink is a case where it could stretch.

Q15. How does the ack timing differ from the SPI demo?

A. SPI: the slave had to preload the reply before the transfer, so 'D' appeared one transfer late and the master needed a dummy byte. I2C: the R/W bit announces the read first, so the slave writes DR during the transaction and the master gets 'D' in the same read.

Q16. Why is PE set last and SWRST pulsed first?

A. CR2, CCR and TRISE can only be written while PE = 0, and SWRST returns the peripheral to a known state. Enabling PE last starts the peripheral with a clean configuration.

Q17. What would happen if the slave blinked 3 s instead of 2 s?

A. The master's read at 2.5 s would find the slave busy. The hardware may ACK the address but stretches SCL until firmware serves it, so the master's guard counter would likely expire and the read would fail, with no confirmation blink. The slave could also be left waiting for AF. Fix: poll a status byte or lengthen the wait.

Q18. Name two problems with this design and fix them.

A. (1) Blind 2500 ms delay: poll a status byte or use clock stretching. (2) Slave loops without timeouts and a blocking blink: use interrupts and guard counters or a watchdog.

ONE-PAGE CRAM CARD

Read this last, then sleep

Topic	Remember
Flow	Press -> WRITE 'S' (0x53) to 0x28 -> slave blinks x5 (2000 ms), arms ack -> master waits 2500 ms -> READ 1 byte -> 'D' (0x44) -> master NACK, STOP -> blinks x2
Wiring	PB8 SCL to PB8, PB9 SDA to PB9, 4.7k each to 3.3 V, GND. AF4, open-drain.
Address	0x28 -> 0x50 (write) / 0x51 (read). OAR1 = 0x4050 (bit 14 = 1).
Timing regs	CR2 = 16 (MHz); CCR = 16 MHz/(2 x 100 kHz) = 80; TRISE = 1000 ns/62.5 ns + 1 = 17
CR1 bits	0 PE, 8 START, 9 STOP, 10 ACK, 15 SWRST. Slave CR1 = 0x401.
SR1 flags	0 SB, 1 ADDR, 2 BTF, 4 STOPF, 6 RXNE, 7 TXE, 10 AF
SR2 flags	0 MSL, 1 BUSY, 2 TRA (1 = transmit)
Clearing	ADDR: SR1 then SR2. SB: SR1 then DR write. STOPF: SR1 then CR1 write. AF: write 0. BTF: SR1 then DR.
Masks	MODER 0x000F0000 -> 0x000A0000; OTYPER |= 0x300; PUPDR -> 0x00050000; AFRH 0xFF -> 0x44
Clocks	AHB1ENR bit0/1/2 = GPIOA/B/C; APB1ENR bit21 = I2C1
Key traps	1-byte read: ACK=0 BEFORE clearing ADDR, STOP right after. Slave: SR1 then SR2. Slave read replies on demand.
Weak points	Blind delay, blocking blink, slave loops without timeout, no bus recovery, polling
I2C vs SPI	2 wires + address + ACK, half duplex, open-drain, slower | SPI: full duplex, push-pull, faster, preload reply

7-STEP RECIPE FOR I2C ON STM32 (REGISTER LEVEL)

1) Enable GPIO and I2C clocks. 2) Pins to AF mode. 3) Choose AF4, open-drain, pull-ups. 4) SWRST pulse. 5) Set CR2 (FREQ), CCR, TRISE, OAR1 (slave) while PE = 0. 6) Set PE (and ACK for slave). 7) Transfer by following the event flags (EV5, EV6, EV8 ... for master; EV1, EV2/EV3, EV4 for slave).

Mnemonics: **SCL** = the Clock beat; **SDA** = the Data words. **0 = Write, 1 = Read** in the R/W bit (Write has the zero, like 'nothing returned'). **ACK is LOW**: a nod pulls the wire down. **Read SR1 then SR2** to clear ADDR.